

Part 3 – Testing and Validation

Introduction

This report analyzes the testing and validation strategy for the ASU ROAR autonomous rover system described in the case study. The rover is designed for the European Rover Challenge in Poland, where it must operate on an outdoor Mars-analogue terrain under changing weather, lighting, terrain, and temperature conditions. The system architecture is based on ROS 2 and micro-ROS, with major modules including perception, localization, path planning, control, and a mission manager. The rover uses sensors such as an RGB-D camera, IMU, wheel encoders, LiDAR, and current sensors, while low-level actuation is handled through an embedded microcontroller connected through a micro-ROS bridge.

1 Q1 – Verification and Validation

Verification and validation are both required to build confidence in an autonomous system, but they answer different questions.

Verification asks whether the system was built correctly according to its design requirements. It focuses on checking that each module, interface, and algorithm behaves as specified. For example, in the ASU ROAR rover, verification includes checking that the RGB-D camera publishes valid image topics, the IMU publishes acceleration and angular velocity at the expected rate, the wheel encoders produce reasonable odometry values, and the micro-ROS bridge correctly transfers actuator commands between the ROS 2 computer and the STM32 microcontroller. Another example is verifying that the A* global planner produces a collision-free path when given a known occupancy grid.

Validation asks whether the correct system was built for the real mission environment. It focuses on whether the rover can actually complete the ERC tasks under realistic outdoor conditions. For example, even if the ArUco detection node is verified to work under laboratory lighting, validation requires testing whether it still detects markers in the Marsyard when there is direct sun glare, shadow changes, or low-contrast lighting. Similarly, the localization system may pass unit tests, but it must be validated by checking whether the rover can still reach checkpoints accurately when wheel slip, loose gravel, or IMU drift are present.

In this project, verification is mainly performed through unit tests, topic checks, simulation replay, interface tests, and module-level tests. Validation is performed through full mission simulations, corner-case scenarios, stress tests, and repeated benchmark runs that represent the real Poland competition environment.

2 Q2 – Limitations and Failure Conditions of Major Modules

Perception

The perception subsystem converts raw sensor data into useful semantic information. It includes ArUco marker detection, LiDAR-based obstacle detection, and visual odometry.

Its main limitation is sensitivity to sensor quality and environmental conditions. The RGB-D camera and visual odometry may degrade under direct sunlight, glare, low contrast, shadows, rain, dust, or motion blur. ArUco detection may fail if the marker is partially occluded, too far from the camera, seen at a sharp angle, or washed out by sunlight. LiDAR obstacle detection may degrade if dust, rain, reflective objects, or sparse point clouds produce false positives or false negatives.

The perception module may fail when the occupancy grid does not represent the real terrain accurately. For example, a small rock, step obstacle, or loose gravel patch may not be classified correctly. This can lead the local planner to believe that an unsafe path is clear.

Localization

The localization subsystem fuses wheel odometry, IMU data, visual data, ArUco marker corrections, and possibly visual SLAM. Its purpose is to estimate the rover pose in the world frame.

Its main limitation is drift. Wheel odometry can drift during wheel slip, especially on loose gravel, soft sand, or steep slopes. IMU measurements can accumulate error over time, and the case study specifically warns about IMU thermal drift during temperature changes. Visual odometry can degrade in low-texture, low-contrast, overexposed, or shadow-heavy scenes. ArUco localization helps correct drift, but only when markers are visible and correctly detected.

Localization may fail when multiple sensors degrade at the same time. A difficult corner case is a steep sandy slope under poor lighting, where the wheels slip, the camera loses visual features, and the IMU experiences bias. In this case, even an adaptive filter such as Fuzz-IESKF may not have enough reliable information to maintain accurate pose estimation.

Path Planning

The path planning subsystem uses A* for global planning and APF for local reactive planning.

The global planner depends on the correctness of the global occupancy map. If the map is outdated, incomplete, or inaccurate, A* may produce a path that is theoretically collision-free but physically unsafe. It may also fail when the map resolution is too low to represent small hazards such as rocks, narrow gaps, or step obstacles.

The local planner reacts to nearby obstacles, but APF-based methods can suffer from local minima. For example, the rover may become stuck between obstacles, oscillate near a narrow passage, or choose an unsafe direction if repulsive and attractive forces balance poorly. The local planner can also degrade when obstacle data is noisy, delayed, or inconsistent.

Control

The control subsystem converts planned trajectories into actuator commands. It includes adaptive pure pursuit, velocity profiling, arm control, and micro-ROS actuation.

Its main limitation is the difference between commanded motion and actual rover motion. On loose gravel, sand, or steep slopes, the rover may not follow the planned trajectory because of wheel slip, limited traction, or actuator saturation. Adaptive pure pursuit may produce tracking errors when the path curvature changes sharply or when the rover speed is too high for the terrain. Velocity profiling may also fail if the terrain-dependent speed limits are not conservative enough.

The micro-ROS actuation path introduces another possible limitation. If serial communication delay, packet loss, timing jitter, or actuator saturation occurs, the low-level motor drivers may not execute commands as expected. This can cause delayed response, oscillation, overshoot, or failure to stop in time.

3 Q3 – Environmental Challenges and Corner Cases

The Poland Marsyard environment introduces several challenges that can create difficult corner cases for the autonomous rover.

Weather Conditions

Rain and high humidity can affect camera visibility, LiDAR readings, wheel traction, and electrical reliability. Wet surfaces may become reflective, which can confuse visual perception. Humidity can also increase the risk of sensor noise or connector issues. Wind and dust can reduce camera clarity and may disturb lightweight objects or dust particles around the rover.

A corner case occurs when light rain creates both reduced traction and reflective surfaces. In this case, the rover may experience wheel slip while perception simultaneously receives degraded visual information.

Lighting and Visibility

Direct sun glare, specular reflections, changing shadows, overcast skies, and low-contrast scenes directly affect camera-based perception. Since the rover uses an RGB-D camera for ArUco detection and visual odometry, changing lighting can affect both marker detection and localization.

A difficult corner case occurs when an ArUco marker is visible but partially washed out by glare. The perception system may either fail to detect it or estimate an incorrect pose. If this incorrect pose is injected into localization, the rover may receive a wrong absolute correction.

Terrain Variability

The Marsyard includes steep slopes, loose gravel, irregular rock fields, step obstacles, and soft or sandy soil. These conditions affect mobility, obstacle detection, localization, and control.

A corner case occurs when the rover is climbing a loose gravel slope while turning. The wheels may slip asymmetrically, causing the odometry to report forward progress while the rover moves less than expected or changes heading incorrectly. This affects localization, planning, and control at the same time.

Temperature Extremes

The case study states that early September temperatures in Poland may vary between approximately 8°C and 28°C. Low temperature can reduce battery performance, while temperature changes can cause IMU thermal drift.

A corner case occurs during a long mission run where the system starts in cold conditions and later warms up. The IMU bias may change gradually, making the localization error grow slowly rather than failing suddenly. This type of failure is hard to detect because the system may appear stable at first.

4 Q4 – Simulation Test Scenarios for Corner Cases

Scenario	Subsystem Stressed	Simulated Conditions	Assumption or Failure Mode Probed
Sun glare on ArUco marker	Perception and Localization	Place ArUco markers in the scene with strong directional light, reflections, partial overexposure, and changing camera angle.	Tests the assumption that markers can be detected reliably under outdoor lighting. Probes false negative detection and incorrect marker pose estimation.
Low-contrast navigation scene	Perception and Visual Odometry	Use overcast lighting, low-texture terrain, and reduced contrast between rocks and ground.	Tests whether visual odometry and obstacle detection still work when visual features are weak.
Loose gravel slope climb	Localization and Control	Simulate a sloped terrain with reduced wheel friction and small rocks. Command the rover to climb and turn.	Probes wheel slip, odometry drift, heading error, and controller tracking failure.
Soft sand traversal	Control and Path Planning	Create a sandy region with high rolling resistance and reduced traction.	Tests whether the velocity controller and local planner can handle terrain where commanded velocity does not match actual velocity.
Irregular rock field	Perception and Local Planning	Add uneven rocks, small step obstacles, and narrow passages.	Tests whether LiDAR-based obstacle detection and APF local planning can avoid small hazards and avoid local minima.

Scenario	Subsystem Stressed	Simulated Conditions	Assumption or Failure Mode Probed
Sensor delay and micro-ROS jitter	Control and Communication	Add artificial latency and packet jitter between ROS 2 commands and embedded motor execution.	Tests the assumption that actuator commands are executed with deterministic latency. Probes delayed stopping, oscillation, and path tracking degradation.
IMU thermal drift over long run	Localization	Inject slowly changing IMU bias during a long simulated mission.	Tests whether the localization filter can remain stable when IMU bias changes over time.
Dynamic obstacle near path	Local Planning and Control	Introduce a moving obstacle or suddenly appearing object near the planned path.	Tests the APF local planner response to unexpected obstacles and whether the controller can safely slow down or re-route.

5 Q5 – Stress Testing Strategy

Stress testing pushes each module beyond normal operating conditions to identify its operational limits and early signs of degradation.

Perception Stress Testing

Perception can be stress tested by increasing glare, reducing contrast, adding dust or rain effects, reducing camera frame rate, adding motion blur, and introducing partial marker occlusion. LiDAR stress tests can include sparse point clouds, noisy returns, reflective objects, and small obstacles.

The main indicators to monitor are detection recall, false positive rate, false negative rate, marker pose error, obstacle grid accuracy, camera frame rate, perception node latency, and dropped frames. Failure is indicated by missed obstacles, unstable marker pose estimates, or perception latency exceeding the planner update period.

Localization Stress Testing

Localization can be stress tested by increasing wheel slip, adding IMU bias, reducing visual odometry reliability, removing ArUco marker visibility, and simulating long GPS-denied runs. The system should also be tested with conflicting sensor inputs, such as wheel odometry reporting motion while visual odometry reports low movement.

The main indicators to monitor are localization RMSE, yaw error, covariance growth, filter innovation, pose jump after marker correction, accumulated drift per meter, and transform stability. Failure is indicated by high pose error, unstable TF transforms, rapidly increasing covariance, or sudden incorrect pose corrections.

Path Planning Stress Testing

Path planning can be stress tested using dense obstacle fields, narrow corridors, dead ends, changing obstacle positions, and incomplete maps. The global planner should be tested with large maps and blocked paths, while the local planner should be tested with sudden obstacles and local-minimum configurations.

The main indicators to monitor are planning latency, path length, number of replans, collision rate, minimum obstacle clearance, local planner oscillation, and goal success rate. Failure is indicated by no valid path, excessive replanning, local minima, unsafe clearance, or planner latency that prevents real-time response.

Control Stress Testing

Control can be stress tested by commanding sharp turns, high curvature paths, sudden stops, slope traversal, low-traction surfaces, actuator saturation, and communication jitter through the micro-ROS bridge. The arm controller can be stressed using contact tasks, high load, and current-limit conditions.

The main indicators to monitor are cross-track error, heading error, velocity tracking error, wheel slip ratio, motor current, actuator saturation, command latency, PWM response, and emergency stop response time. Failure is indicated by overshoot, oscillation, inability to track the path, excessive current draw, or unsafe stopping distance.

6 Q6 – Benchmarking Strategy

A benchmarking strategy should allow the team to compare the rover across multiple test runs and software versions in a repeatable way.

First, a fixed set of benchmark scenarios should be defined. These should include nominal driving, marker-based localization, obstacle avoidance, slope traversal, low-contrast perception, loose gravel navigation, and full mission simulation. Each benchmark should specify the world file, robot configuration, sensor noise parameters, lighting settings, terrain friction, start pose, goal pose, and random seed.

Second, each run should be recorded using ROS 2 bag files. The recorded data should include sensor topics, TF, odometry, localization output, occupancy grids, planned paths, velocity commands, motor feedback, diagnostic messages, and mission manager states. A metadata file should also be stored for each run, including software version, Git commit hash, parameter files, simulation world version, test date, and operator notes.

Third, results should be processed using an automated evaluation script. The script should calculate metrics such as localization RMSE, detection recall, planning latency, path tracking error, collision count, mission completion time, and success/failure status. The results should be stored in a structured format such as CSV, JSON, or a database.

Finally, performance should be compared across runs using tables and plots. A new software version should only be accepted if it improves or maintains key metrics without causing regressions. For example, a perception update should not reduce ArUco detection recall, and a planner update should not increase collision rate or planning latency.

7 Q7 – Quantitative Metrics

Module	Metric	Measurement Method	Acceptable Threshold	Tracking Method
Perception	ArUco detection recall	Compare detected markers against ground-truth visible markers in simulation.	At least 90% recall in normal lighting and at least 75% in difficult lighting.	Store per-run recall in CSV and plot across Git commits.
Perception	Obstacle detection false negative rate	Compare generated occupancy grid against ground-truth obstacle map.	Less than 5% missed critical obstacles.	Track from rosbag replay and automated map comparison.
Localization	Position RMSE	Compare estimated rover pose against simulation ground truth.	Less than 0.25 m during normal runs and less than 0.50 m in degraded terrain.	Store RMSE per benchmark run and compare across versions.
Localization	Yaw error	Compare estimated heading against ground truth.	Less than 5 degrees in normal operation.	Plot heading error over time from recorded TF and ground truth.

Module	Metric	Measurement Method	Acceptable Threshold	Tracking Method
Path Planning	Planning latency	Measure time between goal/map update and valid path output.	Global planning below 500 ms; local planning below 100 ms.	Log planner timestamps and summarize average and worst-case latency.
Path Planning	Minimum obstacle clearance	Measure closest distance between planned path and obstacles.	At least 0.30 m clearance unless operating in a narrow-passage test.	Compute from occupancy grid and planned path after each run.
Control	Cross-track error	Compare actual rover trajectory against planned path.	Mean error below 0.20 m; maximum error below 0.50 m.	Calculate from trajectory logs and store with software version.
Control	Velocity tracking error	Compare commanded velocity against measured wheel/ground velocity.	Mean error below 10% during normal terrain traversal.	Track from command and encoder feedback topics.

Conclusion

The ASU ROAR rover requires a structured testing and validation approach because its autonomy stack depends on tightly coupled perception, localization, planning, and control modules. The Poland Marsyard environment introduces difficult corner cases such as glare, low contrast, rain, loose gravel, steep slopes, soft soil, and temperature-induced sensor drift. Therefore, testing should not only check whether individual modules work in isolation, but also whether the complete ROS 2/micro-ROS system remains reliable under realistic mission conditions. A combination of simulation corner-case testing, stress testing, rosbag-based replay, quantitative metrics, and benchmark tracking provides a practical way to detect failures and prevent software regressions before competition deployment.